



INSTITUTO POLITECNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO



Lista no. 1:

EJERCICIOS DE TUPLAS Y LISTAS EN PYTHON

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Enlace Notebook: [Lista 1: Ejercicios de tuplas y listas en Python](#)

Miércoles, 03 de septiembre, 2025

CONTENIDO

TUPLAS	3
CONSTRUCTORES DE TUPLAS – TUPLE(ITERABLE).....	3
USO DE ÍNDICES CON TUPLAS	4
TUPLAS CON DIFERENTES TIPOS DE DATOS.....	4
<i>Ejercicio 1</i>	5
LISTAS	5
<i>Ejercicio 2</i>	6
CONSTRUCTOR DE LISTA – LITS(ITERABLE)	8
<i>Ejercicio 3</i>	8
<i>Ejercicio 4</i>	10
<i>Ejercicio 5</i>	12
COMPRENSIÓN DE LISTAS	12
<i>Ejercicio 6</i>	13
CONCLUSIONES	14
AGUIRRE LANTO VÍCTOR MANUEL.....	14
GASCA FRAGOSO PEDRO	14
GUEVARA BADILLO ARELI ALEJANDRA	14
MONTIEL TORO ARAEL DE JESÚS.....	14
RAMÍREZ LOZANO GAELE MARTIN.....	14

Tuplas

```

0 s  tupla1 = (1,3,5,7)

print('Acceso a la tupla con iterator')
for i in tupla1:
    print(i)

print('Acceso a la tupla con índices')
for i in range(len(tupla1)):
    print(i,tupla1[i])

```

Acceso a la tupla con iterator
 1
 3
 5
 7
 Acceso a la tupla con índices
 0 1
 1 3
 2 5
 3 7

EXPLICACIÓN:

En el código se define una tupla con los valores 1, 3, 5 y 7.

Primero se imprime el mensaje “Accesa a la tupla con iterator”, y dentro de un ciclo for se van recorriendo los elementos de tupla1 para mostrarlos uno por uno. Cuando termina el recorrido, se imprime el mensaje “Acceso a la tupla con índices”.

En esta parte, con un ciclo for se repite tantas veces como elementos tiene la tupla (en este caso 4). En cada iteración se imprime el índice y después el valor correspondiente dentro de la tupla.

Constructores de tuplas – tuple(iterable)

```

0 s  [2] lista1 = [1, 2, 3]
      tupla2 = tuple(lista1)
      print(tupla2)

(1, 2, 3)

0 s  [3] lista1[0] = 5
      print(lista1)

[5, 2, 3]

0 s  [4] try:
      tupla2[0] = 5
      print(tupla2)
      except:
      print("problema encontrado D:")

problema encontrado D:

```

EXPLICACIÓN:

En la primera sección del código se definen una lista y una tupla cuyo contenido es el mismo (se copia el contenido de la lista en la tupla haciendo uso de tuple(lista1)).

En la siguiente parte se cambia el elemento de la lista en el índice 0 (1) por el número 5.

Al imprimir la lista podemos visualizar como se realizó el cambio.

Sin embargo, en la última parte del código, al intentar realizar este cambio en el elemento en el índice 0 de la tupla nos marca un error.

Esto sucede debido a que las tuplas son una lista ordenada e inmutable de elementos, por lo que una vez definida la tupla ya no se le podrán realizar ningún cambio.

Uso de índices con tuplas

```

# Tupla completa
print('tupla1:\t', tupla1)

# Toma desde el inicio hasta el índice 3
print('tupla1[:3]:\t', tupla1[:3])

# Toma desde el índice 1 en adelante
print('tupla1[1:]\t', tupla1[1:])

# Toma desde el índice 0 hasta el 1
print('tupla1[0:1]:\t', tupla1[0:1])

# Toma desde el índice 1 y el penultimo elemento
print('tupla1[1:-1]:\t', tupla1[1:-1])

# Revierte el orden de la tupla
print('tupla1[::-1]:\t', tupla1[::-1])

# Hace una copia de la tupla
print('tupla1[:]:\t', tupla1[:])

# Toma elementos de dos en 2 de inicio a fin
print('tupla1[::2]:\t', tupla1[::2])

tupla1: (1, 3, 5, 7)
tupla1[:3]: (1, 3, 5)
tupla1[1:]: (3, 5, 7)
tupla1[0:1]: (1,)
tupla1[1:-1]: (3, 5)
tupla1[::-1]: (7, 5, 3, 1)
tupla1[:]: (1, 3, 5, 7)
tupla1[::2]: (1, 5)

```

EXPLICACIÓN:

En este ejemplo se utiliza la tupla `tupla1 = (1, 3, 5, 7)` para mostrar cómo funcionan los índices y el slicing en Python.

Con `tupla1[:3]` se toman los elementos desde el inicio hasta el índice 3 sin incluirlo, mientras que con `tupla1[1:]` se obtienen los valores desde el índice 1 hasta el final.

Si se indica `tupla1[0:1]`, se selecciona únicamente el primer elemento, y con `tupla1[1:-1]` se toman los valores desde la posición 1 hasta el penúltimo.

También es posible invertir el orden de la tupla con `tupla1[::-1]`, hacer una copia completa con `tupla1[:]` o seleccionar elementos de dos en dos con `tupla1[::2]`.

De esta manera, se aprecia que el slicing permite acceder a diferentes partes de la tupla sin modificar su contenido original.

Tuplas con diferentes tipos de datos

```

[6] tupla1 = (1, 3, 5, 7)

#Tupla de mixta (entero, string, tupla, booleano, float)
tup2 = (1,"John", tupla1, True, -23.1)

[7] #Recorrer la lista por elementos
for i in tup2:
    print(i)

1
John
(1, 3, 5, 7)
True
-23.1

# Recorrer haciendo uso de índice
N = len(tup2)

for i in range(N):
    print(tup2[i])

1
John
(1, 3, 5, 7)
True
-23.1

```

EXPLICACIÓN:

En este ejemplo se define una primera tupla **"tupla1"** y posteriormente una tupla mixta **"tup2"**, la cual contiene distintos tipos de datos, posteriormente se hace uso de dos formas distintas de recorrer la tupla.

El primer método usa un bucle **for** directo, donde la variable **i** toma el valor de cada elemento de la tupla y la imprime elemento por elemento.

El segundo método se recorre la tupla mediante el uso de índices, donde primero calculamos la longitud de la segunda tupla **"tup2"** y se le es asignado el valor a la variable **"N"**, posteriormente se hace uso de un ciclo **for** con **range(N)** para acceder a cada posición, y después imprimimos el elemento que se encuentra en dicha posición hasta recorrer el último elemento.

Ejercicio 1

Modifique el bucle anterior para que imprima el tipo de variable que es cada elemento de la tupla.

```
[ ] # Definimos la tupla como en el ejemplo
tupla1 = (1, 3, 5, 7)
tup2 = (1, "John", tupla1, True, -23.1)

# Modificación del bucle para imprimir el tipo de cada elemento
for i in range(len(tup2)):
    print(type(tup2[i]))
```

```
<class 'int'>
<class 'str'>
<class 'tuple'>
<class 'bool'>
<class 'float'>
```

EXPLICACIÓN:

Para este ejercicio se modificó el bucle a la hora de imprimir, donde ahora indicamos mediante **type(tup2[i])** que imprima el tipo de dato de cada elemento dentro de la **tup2** hasta finalizar el recorrido de esta tal como se muestra en la captura.

Listas

```
[9] lista1 = ['Alvaro', 'Daniel', 'Pilar', 'Beatriz']

for i in lista1:
    print(i, end=' ')
print('')
```

```
Alvaro Daniel Pilar Beatriz
```

```
t1 = (1, 'Oleg', 24.5)      #Tupla 1 mixta
l1 = ['Maria', 'Bonita']   #Lista 1 de strings
l2 = [t1, l1]              #Lista 2 con una tupla y una lista
t2 = (l2, 'manzana')       #Tupla 2 con una tupla y un strig
```

```
[11] print('t1=', t1)
print('l1=', l1)
print('l1=', l2)
print('t2=', t2)
```

```
t1= (1, 'Oleg', 24.5)
l1= ['Maria', 'Bonita']
l1= [(1, 'Oleg', 24.5), ['Maria', 'Bonita']]
t2= [(1, 'Oleg', 24.5), ['Maria', 'Bonita']], 'manzana')
```

EXPLICACIÓN:

En la primera sección definimos una lista compuesta de 4 elementos del tipo string ('Alvaro', 'Daniel', 'Pilar' y 'Beatriz').

En la segunda sección definimos 2 tuplas y 2 listas compuestas de distintos tipos de elementos (enteros, flotantes, strings, listas y tuplas).

```
[12] #Accedemos a los elementos
#Para t1
for i in t1:
    print(i)
print('\n')
```

```
1
Oleg
24.5
```

```
[13] #Para l1
for i in l1:
    print(i)
print('\n')
```

```
Maria
Bonita
```

Ahora para visualizar cada uno de los elementos de las listas y las tuplas definidas recorreremos uno por uno cada uno de sus elementos haciendo uso de un ciclo for.

```
[14] #Para l2
for i in l2:
    print(i)
print('\n')
```

```
(1, 'Oleg', 24.5)
['Maria', 'Bonita']
```

```
[15] #Para t2
for i in t2:
    print(i)
print('\n')
```

```
[(1, 'Oleg', 24.5), ['Maria', 'Bonita']]
manzana
```

Ejercicio 2

Accede al anterior de las estructuras complejas

```
[16] #Tupla a la que estaremos accediendo
      print('t2=', t2)

➦ t2= ((1, 'Oleg', 24.5), ['Maria', 'Bonita']), 'manzana')
```

Primero definimos la tupla con la que vamos a estar trabajando, la cual está compuesta por los siguientes elementos según sus índices:

índice[0] → Una lista compuesta por una tupla y una lista.

índice[0][0] → Tupla con 3 elementos.

índice[0][0][0] → 1

índice[0][0][1] → 'Oleg'

índice[0][0][2] → 24.5

índice[0][1] → Lista con 2 elementos.

índice[0][1][0] → 'Maria'

índice[0][1][1] → 'Bonita'

índice[1] → Un string ('manzana').

```
[17] print(t2[0]) #Imprime el elemento en el índice 0 (la lista con una tupla y una lista)
      print(t2[1]) #Imprime el elemento en el índice 1 (un string)

➦ [(1, 'Oleg', 24.5), ['Maria', 'Bonita']]
   manzana
```

El código accede a los elementos de una estructura de datos usando índices.

- En la primera instrucción se pide el elemento que está en el **índice[0]**.
- En la segunda instrucción se pide el elemento que está en el **índice[1]**.

```
[18] print(t2[0][0]) #Imprime el elemento en el índice 0 de la lista t2[0]
      print(t2[0][1]) #Imprime el elemento en el índice 1 de la lista t2[0]

➦ (1, 'Oleg', 24.5)
   ['Maria', 'Bonita']
```

El código accede a los elementos de una estructura de datos usando índices de forma anidada:

- En la primera instrucción se pide el elemento que está en el **índice[0][0]**.
- En la segunda instrucción se pide el elemento que está en el **índice[0][1]**.

```
[19] print(t2[0][0][0]) #Imprime el elemento en el índice 0 de la tupla en el índice 0 de la lista t2[0]
      print(t2[0][0][1]) #Imprime el elemento en el índice 1 de la tupla en el índice 0 de la lista t2[0]
      print(t2[0][0][2]) #Imprime el elemento en el índice 2 de la tupla en el índice 0 de la lista t2[0]
```

```
1
Oleg
24.5
```

El código accede a los elementos de una estructura de datos usando índices de forma anidada:

- En la primera instrucción se pide el elemento que está en el **índice[0][0][0]**, que corresponde al primer valor dentro de la tupla en el índice[0][0].
- En la segunda instrucción se pide el elemento que está en el **índice[0][0][1]**, que corresponde al segundo valor dentro de la tupla en el índice[0][0].
- En la tercera instrucción se pide el elemento que está en el **índice[0][0][2]**, que corresponde al tercer valor dentro de la tupla en el índice[0][0].

```
[20] print(t2[0][1][0]) #Imprime el elemento en el índice 0 de la lista en el índice 1 de la lista t2[0]
      print(t2[0][1][1]) #Imprime el elemento en el índice 1 de la lista en el índice 1 de la lista t2[0]
      print(t2[0][0][2]) #Imprime el elemento en el índice 2 de la tupla en el índice 0 de la lista t2[0]
```

```
Maria
Bonita
24.5
```

El código accede a los elementos de una estructura de datos usando índices de forma anidada:

- En la primera instrucción se pide el elemento que está en el **índice[0][1][0]**, que corresponde al primer valor dentro de la lista ubicada en el índice[0][1].
- En la segunda instrucción se pide el elemento que está en el **índice[0][1][1]**, que corresponde al segundo valor dentro de la lista ubicada en el índice[0][1].
- En la tercera instrucción se pide el elemento que está en el **índice[0][0][2]**, que corresponde al tercer valor dentro de la tupla ubicada en el índice[0][0].

Constructor de lista – list(iterable)

EXPLICACIÓN:

```
[21] vocalTupla = ('a', 'e', 'i', 'o', 'u')
     vocallista = list(vocalTupla)
```

```
print(vocalTupla)
print(vocallista)
```

```
↗ ('a', 'e', 'i', 'o', 'u')
  ['a', 'e', 'i', 'o', 'u']
```

```
[22] alfabeto = 'abcdefghijklmnñopqrstuvwxyz'
     print('alfabeto es un objeto string: ', type(alfabeto))
```

```
↗ alfabeto es un objeto string: <class 'str'>
```

En este caso se define una tupla llamada **vocalTupla** la cual contiene las 5 vocales, y a continuación convertimos la tupla en una lista, mediante la función **list(vocalTupla)**, asignada a **vocallista**, al imprimirlas podemos observar que el contenido es exactamente el mismo, sin embargo, uno es una tupla y estas no tienen la propiedad de ser modificadas, mientras que a la lista le pueden ser agregados, eliminados o modificados los datos dentro de esta.

Para el siguiente ejemplo se crea una variable **alfabeto** la cual contiene todo el alfabeto, posteriormente imprimimos el tipo de dato al cual pertenece mediante **type(alfabeto)**, arrojándonos que es un objeto de clase string.

```
[23] alfaL = []
```

```
for i in alfabeto:
    alfaL.append(i)
```

```
print(alfaL)
```

```
↗ ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'ñ', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```

En esta parte del código se define una lista vacía (alfaL) y se le agregan cada uno de los elementos de la lista alfabeto uno por uno haciendo uso de **append()** dentro de un ciclo **for**.

Ejercicio 3

a) Concatena 2 listas

```
[24] #Creamos lista del mismo tamaño que alfaL
     N = len(alfaL)
     numL = []
```

```
#llenamos esa lista con numeros (0-25)
for i in range(N):
    numL.append(i)
```

```
print('alfaL = ', alfaL)
print('numL = ', numL)
```

```
↗ alfaL = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'ñ', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
  numL = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26]
```

En esta sección del código creamos una nueva lista llamada **numL** vacía del mismo tamaño que la lista **alfaL**. Posteriormente la llenamos de números empezando desde el 0 y terminando en el 26 ya que la lista es de tamaño 27, igual que **alfaL**.


```
[25] #Creamos una lista que contiene los elementos de alfaL seguidos de los elementos de numL
    alfaNumL = alfaL + numL
    print('alfaNumL = ', alfaNumL)

    #Creamos una lista que contiene la lista alfaL y la lista numL
    alfaNumLL = [alfaL, numL]
    print('alfaNumLL', alfaNumLL)
```

```
alfaNumL = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'ñ', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26]
alfaNumLL [['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'ñ', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'], [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26]]
```

Ahora en la primera parte creamos una nueva lista alfaNumL que contendrá los elementos de alfaL seguido de los elementos de numL, lo que nos da una lista que contiene 54 elementos: a, b, c, ..., y, z, 1, 2, 3, ..., 25, 26.

En la siguiente parte del código creamos otra lista nueva, con la diferencia de que esta está compuesta por dos listas, la lista alfaL y la lista numL, por lo que solo contiene 2 elementos.

b) Accede a los elementos de la lista alfaNumLL

```
[26] for i in range(N):
    print(alfaNumLL[0][i], alfaNumLL[1][i])
```

```
a 0
b 1
c 2
d 3
e 4
f 5
g 6
h 7
i 8
j 9
k 10
l 11
m 12
n 13
ñ 14
o 15
p 16
q 17
r 18
s 19
t 20
u 21
v 22
w 23
x 24
y 25
z 26
```

EXPLICACIÓN:

Ahora recorreremos los elementos de la lista alfaNumLL, esta al conformarse de 2 listas es necesario usar un doble índice para poder acceder a cada uno de los elementos que le corresponde a cada lista.

Hacemos la impresión de datos por pares para poder visualizar los elementos en paralelo:

- alfaL: la recorremos indicando que es el elemento en el índice 0 de la lista principal, y recorremos cada uno de sus elementos aumentando de uno en uno el índice durante el ciclo for.
- numL: la recorremos indicando que es el elemento en el índice 1 de la lista principal, y recorremos cada uno de sus elementos aumentando de uno en uno el índice durante el ciclo for.

c) Eliminar elementos de una lista haciendo uso de lista.remove(objeto)

```
[27] alfaL.remove('c')
    print(alfaL)
```

```
['a', 'b', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'ñ', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
```

Aquí eliminamos el elemento 'c' dentro de la lista alfaL utilizando el método remove().

Una vez impresa la lista nuevamente, podemos ver que el elemento 'c' ya no se encuentra en la lista.

Ejercicio 4

Prueben cada uno de los métodos siguientes en una lista cualquiera e impriman sus resultados.

Method	Description
append()	Agrega un elemento al final de la lista
clear()	Remueve todo los elementos de la lista
copy()	Regresa una copia de la lista
count()	Regresa el número de elementos con el valor especificado
extend()	Agrega elementos de una lista (o cualquier iterable) al final de esta lista
index()	Regresa el índice del primer elemento con el valor especificado
insert()	Adiciona un elemento en la posición especificada
pop()	Remueve un elemento en la posición especificada y puede retornarlo si se hace una asignación
remove()	Remueve el item con este valor específico
reverse()	Invierte el orden de la lista
sort()	Ordena la lista

EXPLICACIÓN:

En este ejercicio lo primero que se tiene que realizar es declarar una lista cualquiera para posteriormente aplicarle todos los métodos de manipulación de listas de la tabla anterior, esto con el objetivo de observar que sucede con la lista si se le aplica alguno de estos.

El primer método es `append()` el cual nos permite agregar un elemento al final de nuestra lista. En este caso como la lista es de nombres se agregó el nombre de "Diego" por lo que el resultado obtenido fue el siguiente ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego'].

El segundo método es `clear()` el cual remueve todos los elementos de nuestra lista. Para poder realizarlo lo que se hizo primero fue hacer una copia de la lista original, esto con el objetivo de no perderla y a la copia realizada se le aplico `clear` dándonos como resultado una lista vacía [].

El tercer método es `copy()` el cual nos regresa una copia de la lista. Para esto se asigna la copia a una nueva variable y así podemos trabajar con ella sin modificar la lista original. El resultado obtenido fue ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego'].

```
[37] #Creamos una lista
nombres = ["Ana", "Luis", "Carlos", "Sofia", "Maria"]
print("Lista original:", nombres)
print()
```

Lista original: ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria']

```
[38] #Método append
nombres.append("Diego")
print("append =", nombres)
print()
```

append = ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego']

```
[40] #Método clear
copiaN = nombres.copy()
copiaN.clear()
print("clear =", copiaN)
print()
```

clear = []

```
[41] #Método copy
nombres_copia = nombres.copy()
print("copy =", nombres_copia)
print()
```

copy = ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego']

```
[42] #Método count
nombres.append("Luis")
countN = nombres.count("Luis")
print("count =", countN)
print()
```

```
count = 2
```

```
[43] #Método extend
nombres2 = ["Fernanda", "Jorge", "Luz"]
nombres.extend(nombres2)
print("extend =", nombres)
print()
```

```
extend = ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz']
```

```
[44] #Método index
indexN = nombres.index("Sofia")
print("index =", indexN)
print()
```

```
index = 3
```

```
[45] #Método insert
nombres.insert(2, "Ricardo")
print("insert =", nombres)
print()
```

```
insert = ['Ana', 'Luis', 'Ricardo', 'Carlos', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz']
```

```
[46] #Método pop
eliminado = nombres.pop(3)
print("Elemento removido:", eliminado)
print("pop =", nombres)
print()
```

```
Elemento removido: Carlos
```

```
pop = ['Ana', 'Luis', 'Ricardo', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz']
```

```
[47] #Método remove
nombres.remove("Luis")
print("remove =", nombres)
print()
```

```
remove = ['Ana', 'Ricardo', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz']
```

El cuarto método es `count()` que nos regresa el número de elementos con el valor que le especifiquemos. Para poder visualizarlo de mejor manera se usó `append` para poder agregar otro nombre que ya existiera en la lista con anterioridad, en este caso "Luis". Una vez hecho lo anterior, se le asignó a una variable el resultado de `count("Luis")`, el cual fue: 2

El quinto método es `extend()`, el cual nos permite agregar varios elementos de otra lista (o de un iterable) al final de la lista principal. Para ejemplificarlo se creó una nueva lista llamada `nombres2` que contenía los valores "Fernanda", "Jorge" y "Luz". Posteriormente, se utilizó `nombres.extend(nombres2)` para añadir dichos elementos a la lista original. El resultado obtenido fue: ['Ana', 'Luis', 'Carlos', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz'].

El sexto método es `index()`, el cual nos regresa el índice del primer elemento con el valor que le hayamos especificado. En este caso se buscó la posición de "Sofía" dentro de la lista `nombres` y el resultado obtenido fue: 3

El séptimo método es `insert()`, el cual nos permite agregar un elemento en una posición específica de la lista. Para este ejemplo se indicó que en la posición 2 se insertara el nombre "Ricardo". De esta manera, el nuevo elemento se acomoda en la posición indicada y los demás se recorren una posición hacia la derecha. El resultado obtenido fue: ['Ana', 'Luis', 'Ricardo', 'Carlos', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz'].

El octavo método es `pop()`, el cual elimina un elemento de la lista en la posición que le indiquemos. Además, este método puede regresar el valor eliminado si se guarda en una variable. En este caso, se eliminó el elemento en la posición 3 y el resultado obtenido fue:

Elemento removido: Carlos y nos dio la siguiente lista: ['Ana', 'Luis', 'Ricardo', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz'].

El noveno método es `remove()`, el cual remueve un elemento de la lista que tengan el valor especificado. Para este caso se eliminó el primer elemento que coincidiera con el nombre "Luis" y el resultado obtenido fue: ['Ana', 'Ricardo', 'Sofia', 'Maria', 'Diego', 'Luis', 'Fernanda', 'Jorge', 'Luz'].

```
[48] #Método reverse
nombres.reverse()
print("reverse =", nombres)
print()
reverse = ['Luz', 'Jorge', 'Fernanda', 'Luis', 'Diego', 'Maria', 'Sofia', 'Ricardo', 'Ana']
```

El décimo método es `reverse()`, el cual nos invierte el orden de la lista. En este caso solo se le dio la lista al método y nos dio como resultado nuestra lista invertida: ['Luz', 'Jorge', 'Fernanda', 'Luis', 'Diego', 'Maria', 'Sofia', 'Ricardo', 'Ana'].

```
[49] #Método sort
nombres.sort()
print("sort =", nombres)
print()
sort = ['Ana', 'Diego', 'Fernanda', 'Jorge', 'Luis', 'Luz', 'Maria', 'Ricardo', 'Sofia']
```

El undécimo método y último es `sort()`, el cual nos ordena la lista. Para este caso como nuestra lista es de nombre nos lo ordeno de manera alfabética dándonos como resultado: ['Ana', 'Diego', 'Fernanda', 'Jorge', 'Luis', 'Luz', 'Maria', 'Ricardo', 'Sofia'].

Ejercicio 5

Declaren una lista vacía, modifiquen el siguiente bucle para que en lugar de imprimir la pareja especificada forme una tupla con ella y la agregue a la lista previamente declarada.

```
[ ] vacia = []

for i in range(len(alfanumLL[0])):
    vacia.append((alfanumLL[0][i], alfanumLL[1][i]))

print(vacia)
```

EXPLICACIÓN:

En este ejercicio se declara una lista vacía llamada `vacia`. Posteriormente, a través de un ciclo `for`, se recorre el rango correspondiente a la longitud de la lista `alfanumLL[0]`. En cada iteración se construye una tupla que contiene el elemento de la primera lista (letra) junto con el elemento correspondiente de la segunda lista (número), y dicha tupla se agrega a la lista `vacia` mediante el método `append`. Al concluir la ejecución, la lista resultante contiene todas las parejas de letras y números organizadas de forma ordenada.

```
[('a', 0), ('b', 1), ('d', 2), ('e', 3), ('f', 4), ('g', 5), ('h', 6), ('i', 7),
```

```
('j', 8), ('k', 9), ('l', 10), ('m', 11), ('n', 12), ('ñ', 13), ('o', 14), ('p', 15),
```

```
('q', 16), ('r', 17), ('s', 18), ('t', 19), ('u', 20), ('v', 21), ('w', 22), ('x', 23), ('y', 24), ('z', 25)]
```

Comprensión de listas

```
[30] frutas = ["manzana", "kiwi", "guanabana", "limón", "níspero", "Pomelo", "Mango"]
lista2 = []
```

EXPLICACIÓN:

Primero creamos nuestras listas: la primera contendrá 7 elementos del tipo `string` ("manzana", "kiwi", "guanabana", "limón", "níspero", "Pomelo", "Mango"). Y la segunda será una lista vacía.

```
#Agrega a la lista2 todas las frutas de la lista frutas que contengan la leta 'a'
for x in frutas:
    if "a" in x:
        lista2.append(x)

print(lista2)

['manzana', 'guanabana', 'Mango']
```

Ahora agregamos a la `lista2` todos los elementos los cuales contengan la letra 'a' dentro de ellos haciendo uso de la condicional `if "a" in x` y del método `append` dentro del ciclo `for` que recorre toda la lista.

```
[31] #Itera sobre la lista frutas y si un elemento (x) contienen una 'a' lo agrega a la lista2
      lista2 = [x for x in frutas if "a" in x]
      print(lista2)

[32] #Itera sobre la lista de frutas y si un elemento NO contiene una 'a' lo agrega a la lista2
      lista2 = [x for x in frutas if "a" not in x]
      print(lista2)

[33] #Itera sobre la lista frutas y si un elemento tiene un largo de menos de 6 caracteres lo agrega a la lista2
      lista2 = [x for x in frutas if len(x) < 6]
      print(lista2)

[34] #Recorre numeros del 0-9 y agrega todos aquellos cuyo modulo2 se igual a 0 (pares)
      lista2 = [x for x in range(10) if x % 2 == 0]
      print(lista2)

[35] #Recorre la tupla t1 y agrega todos aquellos elementos cuyo tipo sea entero o flotante.
      lista2 = [x for x in t1 if type(x) == int or type(x) == float]
      print(lista2)
```

En el bloque 31 de igual forma se agregan todos los elementos con la letra 'a' pero haciendo uso de otra notación.

En el bloque 32 se recorre la lista frutas y se agregan únicamente los elementos que no contienen la letra 'a'.

En el bloque 33 Se recorre la lista frutas y se agrega solo los elementos cuyo largo es menor a 6 caracteres.

En el bloque 34 se genera una lista de números del 0-9 usando range(10) y se incluyen únicamente los elementos cuyo residuo al dividir entre 2 sea cero, es decir, los números pares.

En el bloque 35 se recorre la tupla t1 y se agregan únicamente los elementos cuyo tipo sea entero o flotante.

Ejercicio 6

Usando comprensión de listas, cree una lista que solo incluya los elementos de frutas que tengan más de cinco letras

```
[ ] frutas = ["manzana", "kiwi", "guanabana", "limón", "níspero", "Pomelo", "Mango"]
      lista2 = [x for x in frutas if len(x) > 5]
      print(lista2)
```

EXPLICACIÓN:

En este ejercicio se utiliza una comprensión de listas para filtrar los elementos de la lista frutas. La condición aplicada es que la longitud de cada palabra sea mayor a cinco caracteres, por lo que solo se agregan a la nueva lista aquellos nombres de frutas que cumplen con ese requisito. Al final, la lista resultante contiene ['manzana', 'guanabana', 'níspero', 'Pomelo'], que son las frutas con más de cinco letras en su nombre.

Conclusiones

Aguirre Lanto Víctor Manuel

El trabajo con tuplas y listas en Python me permitió comprender sus diferencias y la forma en que cada estructura puede aprovecharse en distintos contextos. Las tuplas, al ser inmutables, brindan seguridad en los datos, mientras que las listas ofrecen flexibilidad por su capacidad de modificación. Un punto que me resultó más complejo fue el uso de la subindexación o slicing, aunque con la práctica logré asimilar mejor su funcionamiento y utilidad.

Gasca Fragoso Pedro

Al realizar estos ejercicios de repaso logre comprender mejor la diferencia entre las tuplas y las listas, ya que las primeras son estructuras inmutables y las segundas son mutables, lo que permite modificarlas con los diferentes métodos vistos según sea necesario. Además, el uso de índices, métodos y comprensiones de listas me pareció algo muy interesante ya que nos permitió manipular los datos de forma más sencilla y eficiente. También note como la comprensión de listas ayuda a simplificar el código y hacerlo más claro. Considero que estos ejercicios de repaso me ayudaron a reforzar mis habilidades en el manejo de estructuras de datos en Python, lo cual es fundamental para poder resolver futuras prácticas de la unidad de aprendizaje.

Guevara Badillo Areli Alejandra

Gracias a los ejercicios de repaso fue notorio que a pesar de que las listas y las tuplas a simple vista son muy parecidas, en realidad estas cumplen funciones muy distintas. Mientras que las tuplas son más rígidas ya que no se pueden modificar, lo que nos permite mantener datos que necesitamos que sean constantes y sin cambios de forma segura. Por otro lado, las listas al ser mucho mas flexibles pudiendo agregar, quitar o alterar elementos sin problema, nos sirven de mucho para la mayoría de los programas. Así que, esta practica no solo nos ayudo a recalcar las diferencias entre ambas estructuras para saber donde y cuando utilizarlas, sino que también me permitió, de manera personal, un primer vistazo a lo que seria el lenguaje de programación Python.

Montiel Toro Arael de Jesús

En esta práctica de repaso probamos dos estructuras comunes dentro de la programación las cuales permiten al usuario organizar, acceder y manipular datos, destacando que las tuplas garantizan al usuario inmutabilidad y la seguridad del contenido, mientras que las listas ofrecen flexibilidad y distintos métodos de modificación en estas, lo cual nos otorga distintas herramientas para lograr la solución de distintos problemas de manera eficiente, además esta práctica me ayudo retomar algunos conceptos de Python, debido a que solamente hice uso de este en una ocasión para un proyecto, visualizando el por qué es de los lenguajes más utilizados en la actualidad gracias a su versatilidad y ciertas libertades que no tenemos dentro de otros lenguajes de programación.

Ramírez Lozano Gael Martin

La realización de esta práctica ha sido fundamental para consolidar mi entendimiento sobre dos de las estructuras de datos más importantes en Python: las listas y las tuplas. A través de los ejercicios, pude experimentar de manera directa la diferencia clave entre ambas: la mutabilidad de las listas, que permite una manipulación dinámica de los datos mediante métodos como `append()` o `remove()`, frente a la inmutabilidad de las tuplas, que las convierte en una opción ideal para almacenar datos que no deben alterarse. Además, la exploración de la indexación anidada y la comprensión de listas me pareció especialmente útil, ya que simplifica la creación y filtrado de datos complejos, haciendo el código más legible y eficiente. Esta práctica ha reforzado la importancia de elegir la estructura de datos adecuada según las necesidades específicas de cada problema.